

Auth & Session Security Analyzer

Rapport d'audit de securite Android

Application analysee : app-debug
Type d'authentification : SESSION_COOKIE
Vulnerabilites detectees : 46
Date d'analyse : 17/05/2026 13:41

Score global de securite : 0 / 10 (CRITIQUE)

1. Resume Executif

Ce resume presente les principales vulnerabilites identifiees lors de l'audit, evalue le niveau de risque global et formule les recommandations prioritaires.

L'application Android analysée, InsecureBankv2, est une application bancaire mobile délibérément vulnérable utilisant un mécanisme de gestion de sessions par cookies (SESSION_COOKIE) pour l'authentification de ses utilisateurs. L'audit a révélé des vulnérabilités critiques multiples mettant en danger l'intégralité de la chaîne de sécurité : les identifiants (nom d'utilisateur et mot de passe) sont transmis en clair via HTTP sans aucun chiffrement, rendant chaque authentification interceptable par un attaquant positionné en Man-in-the-Middle, et des endpoints sensibles comme /getaccounts sont accessibles sans authentification valide, exposant directement les données bancaires des utilisateurs. Le niveau de risque global est évalué comme critique, avec des scores MASVS de 0/10 pour les domaines Authentification et Réseau, et 2/10 pour le Stockage et la Cryptographie, traduisant une absence quasi-totale de contrôles de sécurité fondamentaux. L'impact potentiel est sévère : un attaquant peut capturer les identifiants en transit, accéder librement aux comptes sans authentification, mener des attaques par force brute sans restriction, et exploiter des sessions non expirées faute de mécanisme de déconnexion côté serveur. Les trois recommandations prioritaires sont les suivantes : premièrement, migrer immédiatement l'ensemble des communications vers HTTPS avec TLS 1.2 minimum et activer le certificate pinning ; deuxièmement, implémenter une gestion de session robuste côté serveur incluant invalidation au logout, rotation des tokens et timeout d'inactivité ; troisièmement, supprimer tout secret codé en dur et toute journalisation d'identifiants, puis déployer une protection anti-force brute sur les endpoints d'authentification.

2. Analyse Statique - Vulnerabilites detectees

L'analyse statique consiste en la decompilation de l'APK avec JADX et l'examen des sources Java/Kotlin par patterns regex. Les resultats sont classes par severite : CRITIQUE (-3 pts), ELEVE (-2 pts), MOYEN (-1 pt).

Fichiers analyses : 2918 | Vulnerabilites : 42 | Score : 0/10

Severite	Type	Fichier	Detail
CRITIQUE	Secret code en dur	AccessibilityNodeInfoCo...	password: "); builder.append(isPassword()); builder.app...
CRITIQUE	Secret code en dur	ChangePassword.java	password=" + this.uname); this.textView_Username = (Text...
ELEVE	Identifiants dans les logs	DoLogin.java	Log.d("Successful Login:", ", account=" + DoLogin.this.username ...
ELEVE	Hachage MD5 faible	zza.java	MessageDigest.getInstance("MD5"
MOYEN	Communication en clair (HTTP)	AnalyticsReceiver.java	http://g
MOYEN	Communication en clair (HTTP)	CampaignTrackingRecei...	http://g
MOYEN	Communication en clair (HTTP)	CampaignTrackingRecei...	http://g
MOYEN	Communication en clair (HTTP)	Tracker.java	http://h
MOYEN	Communication en clair (HTTP)	zza.java	http://g
MOYEN	Communication en clair (HTTP)	zzl.java	http://g
MOYEN	Communication en clair (HTTP)	zzl.java	http://g
MOYEN	Communication en clair (HTTP)	zzl.java	http://g
MOYEN	Communication en clair (HTTP)	zzl.java	http://g
MOYEN	Communication en clair (HTTP)	zzl.java	http://g
MOYEN	Communication en clair (HTTP)	zzl.java	http://g
MOYEN	Communication en clair (HTTP)	zzl.java	http://g

MOYEN	Communication en clair (HTTP)	zzl.java	http://g
MOYEN	Communication en clair (HTTP)	zzl.java	http://g
MOYEN	Communication en clair (HTTP)	zzy.java	http://w
MOYEN	Communication en clair (HTTP)	Action.java	http://s
MOYEN	Communication en clair (HTTP)	Action.java	http://s
MOYEN	Communication en clair (HTTP)	Action.java	http://s
MOYEN	Communication en clair (HTTP)	Action.java	http://s
MOYEN	Communication en clair (HTTP)	Action.java	http://s
MOYEN	Communication en clair (HTTP)	Action.java	http://s
MOYEN	Communication en clair (HTTP)	Action.java	http://s
MOYEN	Communication en clair (HTTP)	Action.java	http://s
MOYEN	Communication en clair (HTTP)	Action.java	http://s
MOYEN	Communication en clair (HTTP)	Action.java	http://s
MOYEN	Communication en clair (HTTP)	Action.java	http://s
MOYEN	Communication en clair (HTTP)	Action.java	http://s
MOYEN	Communication en clair (HTTP)	Action.java	http://s
MOYEN	Communication en clair (HTTP)	Action.java	http://s
MOYEN	Communication en clair (HTTP)	zzm.java	http://p
ELEVE	Hachage MD5 faible	zzak.java	MessageDigest.getInstance("MD5"
ELEVE	Hachage MD5 faible	zzbl.java	MessageDigest.getInstance("MD5"
MOYEN	Communication en clair (HTTP)	zzgk.java	http://w
ELEVE	Hachage MD5 faible	zzhl.java	MessageDigest.getInstance("MD5"
MOYEN	Communication en clair (HTTP)	PlusOneButton.java	http://s
MOYEN	Communication en clair (HTTP)	PlusOneButton.java	http://s
MOYEN	Communication en clair (HTTP)	zzax.java	http://h

3. Analyse Dynamique - Tests d'authentification

L'analyse dynamique evalue le comportement de l'application en cours d'exécution. Les tests portent sur la validité des tokens après déconnexion, la durée de vie, la rotation des tokens et la robustesse du mécanisme d'authentification.

Mode d'analyse :	API_DIRECT
Déconnexion effective :	Non (VULNERABLE - jeu possible)
Durée de vie du token :	None min
Rotation des refresh tokens :	Non testé

Vulnérabilités détectées lors des tests :

CRITIQUE	POST http://localhost:8888/login envoie username+password sans chiffrement - interceptable pa
CRITIQUE	/getaccounts répond HTTP 200 sans mot de passe valide

ELEVE	10 tentatives de login echouees consecutives acceptees sans blocage ni CAPTCHA
MOYEN	L'application n'a pas de mecanisme de logout cote serveur. Le menu 'Restart' ne fait que navigu

4. Scores MASVS par Domaine

Les scores reflètent le niveau de sécurité par domaine OWASP MASVS v2. Chaque vulnérabilité déduit des points selon sa sévérité. Un score inférieur à 4 indique un risque critique sur ce domaine.

MASVS-AUTH		0/10
MASVS-STORAGE		2/10
MASVS-CRYPTO		2/10
MASVS-NETWORK		0/10

5. Checklist de Sécurité (MASVS)

Checklist générée par IA (Claude) adaptée au type d'authentification détecté. Chaque contrôle est mappe à un chapitre OWASP MASVS pertinent. Ces points constituent le référentiel de vérification pour la revue de code.

- [] [MASVS-AUTH-1] Vérifier que l'authentification par SESSION_COOKIE impose un transport exclusivement chiffré (HTTPS/TLS) pour la transmission des identifiants et du cookie de session, sans aucun fallback HTTP
- [] [MASVS-AUTH-2] Vérifier que le cookie de session (JSESSIONID) est invalidé côté serveur lors de la déconnexion explicite de l'utilisateur, rendant le token inutilisable après logout
- [] [MASVS-AUTH-3] Vérifier que le mécanisme d'authentification implémente une protection contre les attaques par force brute (blocage temporaire, CAPTCHA, limitation du taux de requêtes) après un nombre défini de tentatives échouées
- [] [MASVS-AUTH-4] Vérifier que le cookie de session possède un délai d'expiration (timeout) défini côté serveur et que les sessions inactives sont automatiquement invalidées après une période raisonnable
- [] [MASVS-AUTH-5] Vérifier qu'une nouvelle session (rotation de cookie) est générée après chaque authentification réussie afin de prévenir les attaques par fixation de session
- [] [MASVS-AUTH-6] Vérifier que les endpoints sensibles (ex. : /getaccounts) requièrent impérativement une session authentifiée valide et retournent HTTP 401/403 en l'absence de cookie valide
- [] [MASVS-NETWORK-1] Vérifier que toutes les communications réseau de l'application utilisent exclusivement TLS 1.2 ou supérieur, et qu'aucune URL en HTTP clair n'est présente dans le code applicatif métier
- [] [MASVS-NETWORK-2] Vérifier que la configuration Network Security Config interdit explicitement le trafic en clair (cleartextTrafficPermitted="false") et définit un certificate pinning pour les domaines critiques
- [] [MASVS-CRYPTO-1] Vérifier qu'aucun algorithme de hachage faible (MD5, SHA-1) n'est utilisé pour des opérations liées à la sécurité (intégrité des sessions, stockage de mots de passe, signatures)
- [] [MASVS-CRYPTO-2] Vérifier qu'aucun secret, mot de passe ou clé cryptographique n'est codé en dur dans le code source ou les ressources de l'application
- [] [MASVS-STORAGE-1] Vérifier que les cookies de session sont stockés avec les attributs Secure, HttpOnly et SameSite=Strict, et ne sont jamais persistés dans des fichiers journaux, SharedPreferences non chiffrées ou bases de données locales
- [] [MASVS-STORAGE-2] Vérifier qu'aucune information d'identification sensible (nom d'utilisateur, mot de passe, token de session) n'est écrite dans les logs système (Logcat) en environnement de production

- [] [MASVS-AUTH-7] Vérifier que le flux de déconnexion ("logout") détruit effectivement la session côté serveur et redirige l'utilisateur vers l'écran de connexion en effaçant les données de session locales
- [] [MASVS-NETWORK-3] Vérifier que le cookie de session n'est jamais transmis via des paramètres d'URL (query string) susceptibles d'être capturés dans les journaux de proxy ou les en-têtes Referer

6. Criteres d'Acceptation de Securite

Les criteres d'acceptation definissent les conditions que le systeme doit satisfaire pour etre considere comme securise. Ils servent de base aux tests d'acceptation, aux revues de code et aux audits de conformite MASVS.

- 1.** Le système DOIT transmettre l'ensemble des identifiants et cookies de session exclusivement via HTTPS avec TLS 1.2 minimum afin de prévenir toute interception par attaque de type Man-in-the-Middle.
- 2.** Le système NE DOIT PAS autoriser l'accès à un endpoint authentifié (tel que /getaccounts) sans présentation d'un cookie de session valide et actif afin d'empêcher tout accès non autorisé aux données utilisateur.
- 3.** Le système DOIT invalider définitivement le cookie de session côté serveur lors de chaque opération de déconnexion afin de garantir qu'un token capturé ne puisse pas être réutilisé.
- 4.** Le système DOIT générer un nouveau cookie de session après chaque authentification réussie afin de neutraliser les attaques par fixation de session (session fixation).
- 5.** Le système NE DOIT PAS consigner d'informations d'identification sensibles (nom d'utilisateur, mot de passe, token) dans les journaux applicatifs ou système afin d'éviter leur exposition lors d'un accès physique ou logique à l'appareil.
- 6.** Le système DOIT bloquer temporairement ou limiter les tentatives de connexion après un maximum de 5 échecs consécutifs afin de résister aux attaques par force brute ou par dictionnaire.
- 7.** Le système NE DOIT PAS contenir de secrets, mots de passe ou clés cryptographiques codés en dur dans le code source ou les ressources compilées afin d'éviter leur extraction par ingénierie inverse.
- 8.** Le système DOIT appliquer un délai d'expiration de session (idle timeout et absolute timeout) défini et contrôlé côté serveur afin de limiter la fenêtre d'exploitation d'un cookie de session volé.
- 9.** Le système NE DOIT PAS utiliser des algorithmes cryptographiques obsolètes tels que MD5 ou SHA-1 pour toute opération liée à la sécurité afin de garantir l'intégrité et la confidentialité des données traitées.